

AI Risk Assessment: SOC 2 Control-Gap Agent

System: SOC 2 control-gap review agent, v1.1 **Assessed against:** NIST AI Risk Management Framework 1.0 (AI 100-1) **Author:** Sameer Athili **Date:** 16 September 2026 **Status:** Complete for the system as built. Not a production authorisation.

1. Purpose and scope

1.1 What this document is

This is a risk assessment of an AI agent that I designed and built, written the way I would want one written about a system I had to sign off on. It follows the four functions of the NIST AI Risk Management Framework, and then does something the framework does not ask for, because the framework was not written for this kind of system: it names the four places where NIST AI RMF stops short of what an agent actually needs, and says what I did instead.

The assessment covers the system as it exists in the repository. Every control described here is implemented in code and covered by a test, and section 8 states exactly what was verified and what was not.

1.2 What the system does

The agent reviews a vendor's SOC 2 report against a receiving organisation's own control requirements and records one cited finding per requirement in a gap register. It is given six functions it may call. Four of them read the source document and the standard; one writes a finding to the register; one ends the review. It decides for itself which sections to search, which to read, and what each requirement's verdict should be, and it continues until it declares itself finished or a control stops it.

1.3 Why this system and not a simpler one

A read-only summariser would have been easier to build and far less interesting to govern. The point of choosing a task where the system writes to a register is that the governance question changes from accuracy to authority. A tool that is wrong produces a bad sentence. An agent that is wrong produces a record that someone downstream relies on, and the record does not carry a warning label.

1.4 Intended use

The system is intended to produce a **first-pass** gap register that a qualified reviewer then verifies, with every finding traceable to a quoted passage and a page number so that verification takes seconds rather than a re-read. It is intended to be operated by someone competent to evaluate a SOC 2 report themselves.

1.5 Out-of-scope uses

The following uses are outside the intended use and the controls are not designed to make them safe:

- producing a vendor approval decision, or any output treated as a decision rather than as an input to one
- operating unattended, or with `--approve-all` set, in any context where the register is relied upon
- assessing a report type other than a SOC 2, including ISO 27001 certificates, bridge letters and completed security questionnaires, none of which the system prompt or the requirement schema contemplate
- any use where an absent finding would be read as evidence that no gap exists, for the reason given at R-07 in section 4

1.6 Deployment limitation

The agent runs as a command line program on a local machine against a synthetic vendor report. It has never been run against a real SOC 2 report, and the report in the repository is fictional with its gaps deliberately planted. The recorded session shipped with the repository is one path through that one document. It demonstrates the loop and the controls and is not evidence of live model behaviour across documents. No claim about accuracy is made anywhere in this document, because none has been measured.

2. System description (MAP)

2.1 Architecture

The system is a loop of roughly a hundred lines. The program opens a conversation with the model and supplies a list of callable functions with their schemas. The model responds with a request to call one of them rather than with text. The program executes it, returns the result, and asks again. This repeats until the model stops or a control stops it.

Every control in this assessment is applied in the gap between the model requesting a call and the program executing it. That gap is the only place a control can live, which is why the loop is written out explicitly rather than delegated to a framework.

2.2 The action surface

An agent's real capability is whatever its tools can reach, not whatever its prompt says it does. The full surface is six functions:

Function	Tier	Effect
list_report_sections	read	returns section ids, titles, page numbers
read_section	read	returns the full text of one section
search_report	read	keyword search, returns located snippets
get_requirements	read	returns the standard being assessed against
record_finding	write	appends one row to the gap register
finish_review	terminal	ends the run

There is exactly one function that changes state. Everything the system can do to the world outside itself, it does through that one function, which is what makes the authority question tractable.

2.3 Data flow

The source document and the standard are read from local disk. Passages the agent reads are sent to the model provider as part of the conversation. The findings and the audit log are written to local disk. Nothing else leaves the machine, and no data is retained by the system between runs other than the register and the log.

In a real deployment this is the first thing that would need revisiting, because a vendor's SOC 2 report is confidential to that vendor and transmitting passages of it to a third-party model provider is a decision that belongs to the receiving organisation and not to the person running the tool.

2.4 Actors

There are two human roles and they are deliberately distinct. The **operator** runs the agent and answers the approval prompts. The **reviewer** is the person accountable for the gap register as a work product. The system produces a record of what the operator approved; it makes no claim at all about the reviewer, and nothing in the register should be read as a professional opinion.

3. Where NIST AI RMF stops short for an agent

This is the section the framework does not ask for.

NIST AI RMF is a good framework and I used it because its four functions are the right shape for this work. But it was written in January 2023 for systems that produce outputs. An agent takes actions, and four questions follow from that which the framework does not put to you. Each is stated below with what the framework does give, where it stops, and what this system does instead.

3.1 Tool authority

The framework gives you MAP 1 and MAP 2, which establish context and intended use, and GOVERN 1, which asks for policies proportionate to risk.

Where it stops. Nothing in MAP or GOVERN asks you to enumerate the functions the system may invoke. Intended use is a statement about what the system is for; it is not a statement about what the system is able to do. For a model those are nearly the same thing. For an agent they are not related at all: a system whose intended use is “summarise a document” can be given a tool that deletes a table, and no part of the framework will notice.

What this system does. Every callable function is declared in one place with a tier attached, and any call to anything else is refused and logged (AC-1). The declared surface in section 2.2 is the same object the code enforces, so this document cannot describe a capability the system does not have, or omit one it does.

3.2 Action reversibility

The framework gives you MANAGE 2, which covers responding to and recovering from incidents, and MEASURE 2, which covers tracking negative impacts.

Where it stops. Both are framed around harm that has occurred. Neither asks, before deployment, whether the specific actions the system is able to take can be undone, or who is able to undo them. For a model that question is close to meaningless, because the output is the harm and retracting a sentence is not a technical problem. For an agent it is the first question worth asking.

What this system does. The single write action is reversible by run (AC-6), and the reversal deliberately does not touch the audit log. Undoing an action and erasing the record that it happened are different operations, and only one of them is available to an operator. A system where “rollback” quietly means both is worse than one with no rollback at all.

3.3 The audit trail is of actions, not outputs

The framework gives you MEASURE 2.8 on transparency and MANAGE 4.1 on post-deployment monitoring, and a good deal of guidance on provenance and on documenting inputs and outputs.

Where it stops. The unit of record throughout is the input-output pair. That is the right unit for a model. For an agent the question an auditor asks is not “what did it say” but “what did it do, in what order, on whose authority, and what did it see before it did it”. An input-output log cannot answer any of those, because most of what an agent does never appears in its final output.

What this system does. Every tool request, its arguments, the tier, the control decision, the approval and who granted it, and a digest of the result are appended to a log before the result returns to the model (AC-4). The log is the deliverable, and the register is downstream of it. When a run is reversed, the reversal is another entry.

3.4 Human oversight is a control, and controls fail

The framework gives you GOVERN 3.2 on human oversight roles, and MEASURE 2.10 and MANAGE 2.2 on human-AI configuration.

Where it stops. Human review is treated throughout as a mitigation. It is described as something you put in place, never as a control with its own design, its own failure modes and its own test. Nothing in the framework prompts you to ask what happens when the human oversight step produces an answer the human did not give.

What this system does. Approval is a per-action control with a specified design (AC-2), its suppression is itself logged (AC-2a), and the input it accepts is constrained (AC-2b). Section 7 describes the incident that produced AC-2b, which is the clearest evidence I have that this gap is real rather than theoretical.

3.5 What this is not

None of the above is a criticism of NIST AI RMF for failing to be something it did not set out to be. The framework is deliberately technology-neutral and deliberately durable, and its four functions are the reason this assessment has a sensible shape at all. The point is narrower: an assessment that applies the framework faithfully to an agent, and stops there, will be silent on the four questions that matter most about that agent. Practitioners are closing this gap with their own agent-specific frameworks, and this section is my attempt at the same thing at the scale of one small system.

4. Risk register (MEASURE)

Likelihood and impact are my own judgment on a three-point scale, stated so they can be disagreed with. Impact is assessed against the intended use in 1.4, which is a first-pass register reviewed by a qualified person.

Id	Risk	Likelihood	Impact	Treatment	Residual
R-01	The agent records a finding supported by a quotation that does not appear in the source	Low	High	AC-5 rejects any citation not present verbatim in the document	Low

Id	Risk	Likelihood	Impact	Treatment	Residual
R-02	The agent records a finding against a section it never read, reasoning from search snippets or from prior knowledge of what SOC 2 reports usually say	Medium	High	AC-5 requires the cited section to have been read in the same run	Low
R-03	The agent writes to the register without a human deciding that it should	Low	High	AC-2 requires per-action approval; AC-2a logs any suppression	Medium, because <code>--approve-all</code> exists and removes the control
R-04	The approval step records a decision the operator did not make	Medium	High	AC-2b accepts only an explicit yes or no; anything else re-prompts	Low. See section 7.
R-05	The agent misses a requirement entirely because its search never reaches the relevant section	High	High	AC-8 prevents a run being declared complete with any requirement unassessed; nothing detects a finding that is present but shallow	High. This is the largest residual risk in the system.
R-06	A correct quotation is attached to a wrong conclusion	Medium	Medium	None technical. AC-5 makes the finding checkable in seconds, which is a different claim from making it correct	Medium, accepted, and mitigated only by the reviewer in 2.4
R-07	The absence of a finding is read as evidence that no gap exists	Medium	High	Stated in 1.5 as an out-of-scope use and in the README limitations	Medium, accepted. This is a documentation control, which is the weakest kind.
R-08	The run does not terminate, or costs more than intended	Low	Medium	AC-3 halts on iteration, token or wall-clock ceiling	Low
R-09	Confidential vendor material is transmitted to a third-party model provider without that decision having been taken by the party it belongs to	High in any real deployment	High	None implemented. Named in 2.3 as the first thing requiring resolution before real use	High, unmitigated. The system is not deployable against real reports until this is resolved.
R-10	The audit log is altered or lost	Low	High	AC-4 appends and fsyncs per record, so a crashed run still leaves a complete record to the point of failure	Medium. The log is append-only by convention and not by permission; the process could overwrite it.

Two entries deserve to be read rather than skimmed. **R-05** is high, unmitigated and structural: every control in this system operates on something the agent did, so when the agent does nothing, no control fires. **R-09** is the reason the deployment limitation in 1.6 is a statement about what may be done and not merely about what has been done so far.

5. Controls

Each control is stated, then mapped to its implementation and to the test that proves it. The last column is the one that matters: a control with no test is a claim.

Id	Control	Type	Implementation	Test
AC-1	The agent may invoke only tools on a declared allowlist	Preventive	<code>guardrails.ToolAllowlist</code>	<code>test_allowlist_rejects_unknown_tool</code>
AC-2	State-changing tools require explicit human approval of that specific call	Preventive	<code>guardrails.ActionTier</code>	<code>test_write_tier_requires_approval_and_declining_blocks_the_write</code>
AC-2a	Suppression of approval is recorded rather than hidden	Detective	<code>ActionTier.request</code> , <code>approval_decision</code> log entry	<code>test_auto_approve_is_recorded_as_such</code>
AC-2b	Only an explicit yes or no counts as an answer	Preventive	<code>ActionTier._prompt</code>	three tests, listed in <code>CONTROLS.md</code>
AC-3	The loop halts at a declared iteration, token or time ceiling	Preventive	<code>guardrails.RunBudget</code>	three tests
AC-4	Every call, argument, control decision and approval is logged before the result returns to the model	Detective	<code>audit.AuditLog</code>	<code>test_audit_log_records_every_call_and_survives_reread</code>
AC-5	A finding requires a citation present verbatim in the source, from a section read in the same run	Preventive	<code>guardrails.CitationCheck</code> and <code>Toolbox.record_finding</code>	four tests
AC-6	An operator can reverse everything a run wrote, without altering the log	Corrective	<code>tools.rollback_run</code>	<code>test_rollback_removes_only_that_run</code>

Id	Control	Type	Implementation	Test
AC-7	One finding per requirement, from a closed verdict set, against a known requirement id	Preventive	<code>Toolbox.record_finding</code> and the schema enum	two tests
AC-8	A run cannot be declared complete while any requirement is unassessed	Detective	<code>Toolbox.finish_review</code>	<code>test_cannot_finish_with_requirements_unassessed</code>
AC-9	The schemas shown to the model match the methods that execute	Detective	asserted in test, not enforced at runtime	<code>test_tool_schemas_match_implemented_methods</code>

Twenty-four tests pass at the time of writing. Each corresponds to a claim in this document, so the document cannot drift from the system without something going red, which is the specific failure I was trying to design out.

5.1 What AC-5 does and does not claim

AC-5 is the control doing the most work and the one most easily overstated. It rejects a finding unless the quoted text appears in the source document, compared as a normalised string, and unless the agent read that section earlier in the same run. Both checks are mechanical and neither consults the model.

It does not make the reasoning correct. A real quotation can support a wrong conclusion, and nothing in the system evaluates the step between the quotation and the verdict. The claim the control supports is narrow and worth stating exactly: **a reviewer can verify any finding against the source in seconds**. That is a claim about the cost of checking, not about the rate of being right.

6. Monitoring and decommission (MANAGE)

Since the system is not deployed, these are the thresholds that would apply if it were, stated now so that they are not invented later to fit whatever happens.

Monitor: the proportion of findings a reviewer changes on verification; the number of citation rejections per run, which rising would suggest the model is drifting toward paraphrase; the proportion of runs completed with `--approve-all`; and the proportion of runs that end incomplete.

Switch it off if more than one finding in five is changed on verification, because at that point the register costs more to check than to write; if any finding reaches the register without a valid citation, since that is AC-5 having failed rather than degraded; if the audit log is found to be incomplete for any run; or if the register is used as a decision record rather than as an input, which is a governance failure rather than a technical one and is the most likely of the four.

Decommission by exporting the audit log and every register to the receiving organisation's own retention, revoking the API credential, and recording in the log that the system was retired and on whose instruction.

7. Worked example: the control that produced false evidence

This is the part of the assessment I would want to read first if someone handed it to me.

What happened. The approval prompt in the first version treated any input other than `y` as a decline. While the agent was waiting at a prompt, a command was pasted into the terminal. `input()` consumed the whole line as the answer, it was not `y`, and the system recorded `declined by operator` in the audit log against requirement R-02.

Why it matters more than it looks. The visible effect was a missing finding, which is trivial and self-correcting. The real effect was that the audit log asserted a human decision that no human made. AC-2 exists so that the log can be relied on as evidence of oversight; a failure mode in which it silently manufactures that evidence is worse than the control being absent, because an absent control is visible and a false record is not. Everyone downstream of an audit log treats it as fact.

What changed. AC-2b now accepts only an explicit `y` or `n`. Anything else is not treated as an answer and the prompt repeats. An absent operator, which is what happens when input is piped in and runs out, is recorded as “no operator input available” rather than as a refusal somebody chose, because those are different facts and the log should not conflate them.

How it was found. By running the system, not by reading it. I had reviewed that function and written a test for it, and neither the review nor the test asked what the control does with input that is neither approval nor refusal. This is the second time on my own work that verification against the running system has found something that verification against the document did not, and it is the main reason section 8 exists in this form.

8. Verification

8.1 What was verified, and how

Verified by executing the system and reading the resulting audit log, not by inspecting the source:

1. A finding whose citation does not appear in the source is refused, nothing is written, and the refusal names AC-5 in the log.
2. A finding whose citation is genuine but whose section was not read in that run is refused, and the refusal is distinguishable in the log from the case above.
3. A call to a function outside the allowlist is refused and named as AC-1.
4. An approval granted under `--approve-all` is logged as `mode: auto` with the suppression noted, and the write can still be refused afterwards on the evidence, which appears in the log as an approval followed by a block.
5. Declining two findings leaves the other eight committed, `finish_review` refuses, and the run reports itself incomplete with the declined requirements named.
6. A rollback removes exactly the rows written by the named run and leaves both the other run’s rows and the whole audit log intact.
7. Every citation in the shipped recorded session is an exact substring of the source document, asserted at build time.

8.2 What has not been verified

1. **Accuracy.** No finding produced by a live model has been checked against a qualified reviewer’s own conclusion, on this document or any other. Nothing in this assessment should be read as an accuracy claim.
2. **Behaviour across documents.** The system has been exercised against one synthetic report whose gaps I planted, which makes it a demonstration and not a sample.
3. **Behaviour under a live model.** Most verification above used the recorded session, which is deterministic by design. A live model will retry, reason differently after a refusal, and occasionally do something neither the tests nor I anticipated.
4. **The audit log under adversarial conditions.** It is append-only by convention. Nothing was tested about what happens if the process, or anything else on the machine, rewrites it.
5. **AC-9 between test runs.** Schema and implementation are compared in a test rather than generated from one source, so they can diverge and stay diverged until the suite is next run.

9. Change log

Date	Change
16 Sep 2026	Initial assessment written against v1.0.

Date	Change
16 Sep 2026	AC-2b added after the approval prompt was found to record a decline the operator did not make. Risk R-04 added to the register. Section 7 written.
16 Sep 2026	R-09, transmission of confidential vendor material to a third-party model provider, added to the register as high and unmitigated, having been described in section 2.3 without appearing in the register. Found while checking that every risk named in prose also had a row.

10. Standing limitations

Restated together, because they are easier to discount when they are spread across ten sections.

The system has never seen a real SOC 2 report. Its accuracy has never been measured. Its search is keyword-based, so a requirement discussed in language the query does not contain can be missed entirely, and no control detects a miss. Approval is only as strong as the attention of whoever is answering, and a flag exists that removes it. Confidential material would be sent to a third-party provider in any real use, and that decision has not been taken by anyone entitled to take it.

What the system does support is a narrower claim: that an agent's authority can be declared, constrained, recorded and reversed, and that each of those properties can be demonstrated by running it rather than asserted in a document.